```
        else:
            cur.execute(
                """
                SELECT flight_key, timestamp, lat_e6, lon_e6, altitude_m, speed_cms, heading_de2,
                       vs_cms, squawk, source_ip
                FROM trajectory_samples
                WHERE timestamp >= ?
                ORDER BY timestamp ASC
                LIMIT ?
                """,
                (cutoff_iso, limit),
            )

        out = []
        for r in cur.fetchall():
            out.append(
                {
                    "flightKey": r["flight_key"],
                    "timestamp": r["timestamp"],
                    "latDD": r["lat_e6"] / 1000000.0,
                    "lonDD": r["lon_e6"] / 1000000.0,
                    "altitudeM": r["altitude_m"],
                    "speedCMS": r["speed_cms"],
                    "headingDE2": r["heading_de2"],
                    "verVelocityCMS": r["vs_cms"],
                    "squawk": r["squawk"],
                    "sourceIp": r["source_ip"],
                }
            )
        return out

    def write_trajectory_history_csv(self, output_path, flight_key=None, hours=24, limit=200000):
        rows = self.get_trajectory_history(flight_key=flight_key, hours=hours, limit=limit)
        out_path = Path(output_path)
        out_path.parent.mkdir(parents=True, exist_ok=True)
        headers = [
            "flightKey",
            "timestamp",
            "latDD",
            "lonDD",
            "altitudeM",
            "speedCMS",
            "headingDE2",
            "verVelocityCMS",
            "squawk",
            "sourceIp",
        ]
        with out_path.open("w", newline="", encoding="utf-8") as f:
            writer = csv.DictWriter(f, fieldnames=headers)
            writer.writeheader()
            for row in rows:
                writer.writerow(row)
        return str(out_path), len(rows)

    def _estimate_motion_from_recent_samples(self, flight_key):
        cur = self._conn.cursor()
        cur.execute(
            """
            SELECT timestamp, lat_e6, lon_e6, altitude_m
            FROM trajectory_samples
            WHERE flight_key = ?
            ORDER BY timestamp DESC
            LIMIT 2
            """,
            (str(flight_key).strip().upper(),),
        )
        points = cur.fetchall()
        if len(points) < 2:
            return None

        p_new = points[0]
        p_old = points[1]
        _, dt_new = _parse_ts(p_new["timestamp"])
```